

Projet S3 – 2026/2027

OCR Word Search Solver

Rapport de Première Soutenance



Date : Novembre 2026

Membres du groupe :

Mathis BARBIER
Augustin SOURISSEAU
Rémi JUSSAC
Eliott AVERTY

Table des matières

Introduction Générale **2**

1 Rôle et mission de chaque membre **2**

Mathis BARBIER : Binarisation, Analyse Structurelle et Segmentation Globale 2

Rémi JUSSAC : Interface Graphique (IHM), Prétraitement et Architecture Modulaire 3

Augustin SOURISSEAU : Détection Fine et Extraction des Caractères 3

Eliott AVERTY : Réseau de Neurones et Algorithme de Résolution 3

2 Objectifs, planification et état d'avancement **3**

2.1 Objectifs de la première soutenance 4

2.2 Planification et méthodologie 4

2.3 Bilan de l'avancement réel 4

3 Choix techniques, algorithmiques et difficultés **5**

3.1 Prétraitement et Interface Graphique (Rémi JUSSAC) 5

3.2 Binarisation et Segmentation Globale (Mathis BARBIER) 6

3.3 Détection et Extraction des Lettres (Augustin SOURISSEAU) 8

3.4 Algorithme de Résolution – Solver (Eliott AVERTY) 9

3.5 Fondations du Réseau de Neurones (Eliott AVERTY) 10

Conclusion Générale **11**

Bibliographie et Ressources **12**

Ouvrages et documents de référence 12

Documentations techniques officielles 12

Ressources d'apprentissage et forums 12

Annexes **13**

Introduction Générale

La numérisation croissante des documents physiques soulève un défi technologique majeur : la capacité des machines à interpréter visuellement et sémantiquement des données brutes. C'est dans ce contexte d'ingénierie appliquée que s'inscrit le projet informatique de ce troisième semestre (S3) : la conception et le développement complet d'un logiciel OCR (*Optical Character Recognition*) spécialisé dans la résolution algorithmique de grilles de mots mêlés.

L'objectif de ce projet dépasse la simple reconnaissance basique de caractères. Il requiert la mise en place d'une chaîne de traitement complexe, modulaire et robuste. Notre application doit être capable de prendre en entrée une image numérique, potentiellement altérée par des défauts tels qu'un mauvais contraste ou une inclinaison aléatoire, et de l'analyser pour en comprendre la structure géométrique. Cette compréhension passe par la segmentation automatique de la grille et de la liste de mots, suivie de l'extraction individuelle de chaque caractère. Ces données visuelles isolées ont ensuite vocation à être interprétées par un réseau de neurones artificiels conçu sur mesure, avant qu'un algorithme de résolution mathématique ne parcoure la matrice reconstituée pour y localiser les mots cibles.

Ce document constitue le rapport de la première soutenance de suivi du projet. Il a pour vocation de présenter et de justifier les fondations techniques de notre architecture logicielle. À ce stade intermédiaire du développement, notre attention s'est concentrée sur la maîtrise des briques algorithmiques fondamentales imposées par le cahier des charges. Nous y présentons le développement de l'interface graphique initiale et les opérations de prétraitement visuel, incluant la binarisation dynamique et les rotations mathématiques. Nous exposons également nos méthodes de segmentation spatiale et d'extraction fine des lettres. Parallèlement à ce traitement d'image, ce rapport valide la conception de notre algorithme de résolution en ligne de commande, le *Solver*, ainsi que la validation théorique de notre moteur d'intelligence artificielle à travers la preuve de concept (PoC) d'un réseau de neurones fonctionnel sur une porte logique.

Afin d'exposer clairement notre démarche conceptuelle et technique, ce rapport s'articulera en trois temps majeurs. Dans un premier temps, nous détaillerons l'organisation interne de l'équipe en explicitant les responsabilités et la mission spécifique de chaque développeur. Dans un deuxième temps, nous dresserons le bilan de notre état d'avancement par rapport aux objectifs visés, tout en détaillant notre méthodologie de travail et de versioning. Enfin, la troisième partie, qui constitue le cœur analytique de ce document, sera consacrée à l'explicitation exhaustive de nos choix technologiques et algorithmiques, ainsi qu'à l'analyse des obstacles techniques rencontrés et des solutions apportées pour fiabiliser chaque module de notre chaîne OCR.

1. Rôle et mission de chaque membre

La réalisation du projet OCR Word Search Solver, imposant de concevoir une chaîne de traitement complète allant de l'analyse d'image brute à la résolution algorithmique, a nécessité une répartition stratégique des tâches. Notre groupe s'est organisé autour de quatre pôles de compétences distincts, permettant à chacun de se spécialiser tout en garantissant l'intégration continue des différents modules.

Mathis BARBIER : Binarisation, Analyse Structurelle et Segmentation Globale

Mon rôle principal au sein du groupe a consisté à concevoir les premières étapes du traitement de l'image, cruciales pour la suite de la chaîne de reconnaissance. J'ai pris en charge la conver-

sion de l'image en un format exploitable mathématiquement via une binarisation dynamique (implémentation de la méthode d'Otsu). J'ai ensuite développé les algorithmes d'analyse structurelle reposant sur des profils de projection (histogrammes horizontaux et verticaux). Cette approche m'a permis de segmenter formellement l'image en identifiant sa disposition (paysage ou portrait) et en isolant avec précision les deux zones d'intérêt majeures : la grille de lettres et la liste de mots, notamment grâce à un algorithme de rognage de précision (*Crop Box*). Je me suis également chargé de la coordination, de la synthèse et de la mise en page de ce rapport d'étape.

Rémi JUSSAC : Interface Graphique (IHM), Prétraitement et Architecture Modulaire

Au sein du groupe, je suis principalement chargé du développement de l'interface graphique et d'une partie du prétraitement des images. Ma mission consiste à fournir un environnement fonctionnel permettant à l'utilisateur de charger et manipuler l'image. J'ai développé l'interface en langage C avec GTK3 et géré le chargement des images via SDL2_image. J'ai implémenté la conversion automatique en niveaux de gris (par calcul de luminance) et la rotation manuelle (par pas de 90° et par angle libre via interpolation bilinéaire). Enfin, j'ai veillé à organiser le code en modules indépendants pour faciliter la maintenance et l'intégration des algorithmes développés par le reste de l'équipe.

Augustin SOURISSEAU : Détection Fine et Extraction des Caractères

Une fois les zones principales délimitées par Mathis, ma mission consistait à descendre à l'échelle du caractère. J'étais chargé de détecter plus finement la grille de lettres, puis d'identifier et d'extraire individuellement chaque caractère contenu dans la grille et dans la liste de mots. Pour accomplir cela, j'ai mis en œuvre plusieurs algorithmes capables de s'adapter à deux situations : les grilles comportant un quadrillage explicite et celles ne contenant que du texte. Mon travail a nécessité une gestion fine du bruit numérique et le calcul adaptatif de l'épaisseur des bordures des cellules, afin de générer des images de lettres parfaitement isolées et prêtes à être analysées par le réseau de neurones.

Eliott AVERTY : Réseau de Neurones et Algorithme de Résolution

Mon rôle s'articule autour de deux pôles algorithmiques : développer le réseau de neurones permettant de reconnaître les lettres et concevoir le solveur. Pour le réseau de neurones (*from scratch*), j'ai opté pour une implémentation vectorisée des matrices. J'ai codé la fonction d'initialisation, la fonction sigmoïde, le passage des données (*forward*), le calcul de l'erreur (*backward*) et la mise à jour des poids, aboutissant à une preuve de concept fonctionnelle pour la porte logique NXOR. En parallèle, j'ai développé l'algorithme de résolution de la grille (le solveur CLI) : il s'agit d'une recherche linéaire où le mot saisi est testé direction par direction dans la matrice chargée jusqu'à obtenir ses coordonnées de départ et d'arrivée.

2. Objectifs, planification et état d'avancement

2.1. Objectifs de la première soutenance

Cette première soutenance de suivi vise à démontrer la mise en place de nos fondations techniques et la maîtrise des briques algorithmiques de base. Conformément au cahier des charges, les éléments obligatoires à valider sont les suivants : le chargement d'une image suivi de la suppression de ses couleurs, l'application d'une rotation manuelle, et la détection précise de la position de la grille, de la liste de mots, ainsi que des lettres individuelles qui les composent. L'algorithme doit ensuite découper l'image pour sauvegarder chaque lettre sous forme de fichier indépendant. En parallèle du traitement d'image, nous devons présenter un programme en ligne de commande (solver) capable de résoudre une grille de mots cachés chargée depuis un fichier texte, ainsi qu'une preuve de concept (PoC) d'un réseau de neurones artificiel capable d'apprendre la fonction logique $\overline{A} \cdot \overline{B} + A \cdot B$ (porte NXOR).

2.2. Planification et méthodologie

Pour coordonner le travail simultané de quatre développeurs, le projet s'appuie sur un dépôt de contrôle de version Git partagé. Chaque membre a développé son module sur des branches dédiées avant d'effectuer les fusions vers la branche principale. Le développement respecte strictement les contraintes techniques imposées : le code est écrit en langage C et compile sans erreur avec les options `-Wall -Wextra`. Pour garantir un code lisible et conforme aux normes attendues, la longueur des lignes est strictement limitée à 80 colonnes, sans espaces inutiles en fin de ligne. De plus, l'ensemble des identifiants et des commentaires est rédigé en anglais. Enfin, une attention particulière a été portée à la rigueur des allocations dynamiques et à la libération de la mémoire lors des manipulations de surfaces graphiques et des tableaux, afin d'assurer la stabilité du programme sans avoir recours à des outils de profilage externes dans un premier temps.

2.3. Bilan de l'avancement réel

À ce jour, l'équipe a atteint l'intégralité des objectifs fixés pour cette première soutenance.

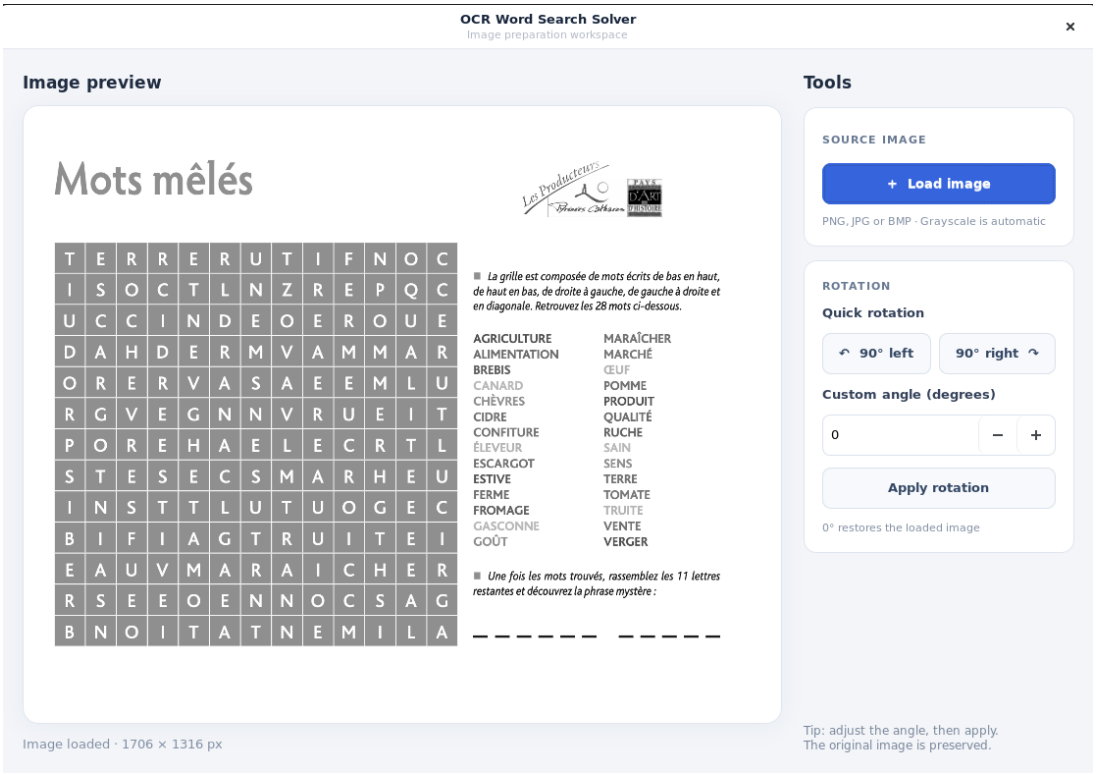
- L'interface graphique construite avec GTK3 permet de charger des images standards (PNG, JPEG, BMP), d'appliquer automatiquement une conversion en niveaux de gris, et d'effectuer des rotations manuelles fluides (par pas de 90° ou par angle libre via interpolation bilinéaire).
- La chaîne de segmentation dynamique (seuillage d'Otsu et analyse d'histogrammes) détecte formellement la grille et la liste de mots, quelle que soit leur disposition relative sur la page.
- L'extraction des caractères s'adapte avec succès aux grilles quadrillées et non-quadrillées en gérant le rognage et le bruit numérique. Le programme découpe l'image et sauvegarde avec succès chaque lettre sous forme de fichier `.png` indépendant dans le dossier `image_storage`. L'algorithme de résolution (CLI) est pleinement fonctionnel : il parcourt la matrice textuelle dans toutes les directions et affiche les coordonnées exactes du mot recherché au format attendu `(x0,y0)(x1,y1)`.
- Le réseau de neurones a été implémenté *from scratch* avec succès, et son algorithme d'apprentissage valide la preuve de concept sur la porte NXOR.

La prochaine phase du projet se concentrera sur la soutenance finale. L'objectif sera de relier ces modules indépendants en un pipeline automatisé, d'implémenter la rotation automatique de l'image (*deskewing*), d'entraîner le réseau de neurones sur un véritable jeu de caractères, et de reconstruire la grille finale résolue directement sur l'interface graphique.

3. Choix techniques, algorithmiques et difficultés

3.1. Prétraitement et Interface Graphique (Rémi JUSSAC)

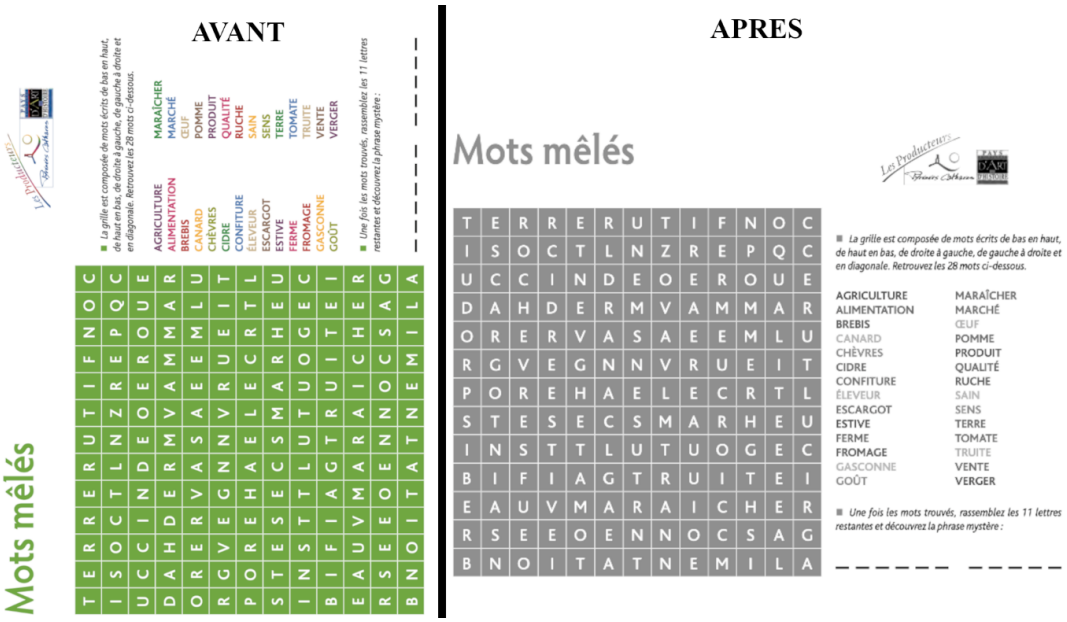
Pour réaliser l'interface graphique requise, j'ai choisi la bibliothèque GTK3, particulièrement adaptée au développement d'applications en C sous Linux. L'application repose sur `GtkApplication` pour la gestion de son cycle de vie, avec un affichage basé sur `GtkDrawingArea` permettant d'adapter l'image à l'espace disponible sans altérer les données sources. Concernant la manipulation des images, j'ai retenu SDL2 et SDL2_image. La fonction `IMG_Load()` charge l'image en mémoire, que je convertis systématiquement au format `SDL_PIXELFORMAT_RGB888` (32 bits, quatre octets par pixel). Ce format standardisé assure la compatibilité avec les algorithmes des autres membres de l'équipe.



La première étape algorithmique de prétraitement consiste en la conversion en niveaux de gris. J'ai implémenté le calcul de luminance classique donnant un poids supérieur au vert :

$$Y = 0.299\,R + 0.587\,G + 0.114\,B$$

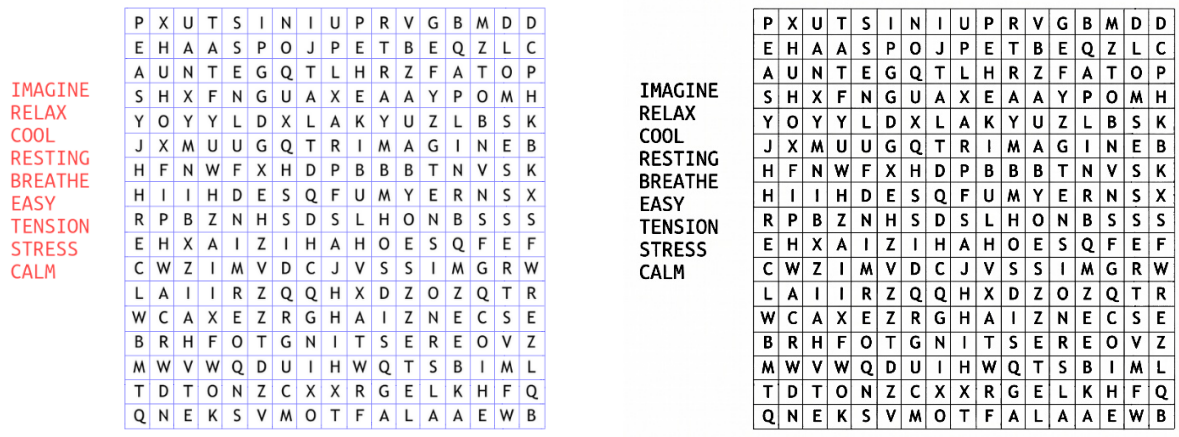
Les trois composantes (R, G, B) prennent ensuite cette valeur Y. Pour redresser l'image, j'ai développé deux méthodes. La rotation par pas de 90° utilise une transformation directe des coordonnées ($x' = H - 1 - y$ et $y' = x$), tandis que la rotation libre selon un angle défini par l'utilisateur repose sur les fonctions trigonométriques inverses et une interpolation bilinéaire pour lisser les contours obliques.



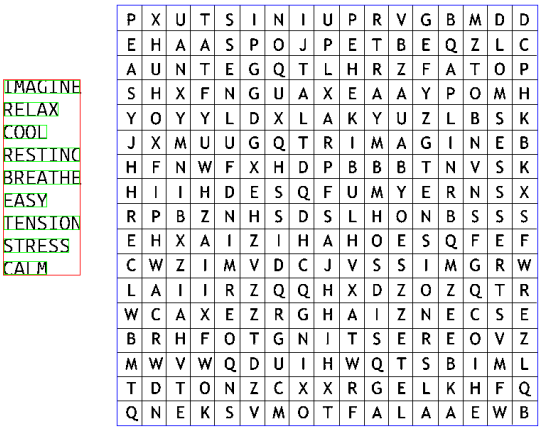
La principale difficulté rencontrée a été de faire communiquer GTK3 (affichage) et SDL2 (traitement), résolue par le développement d'une couche de conversion indépendante. De plus, pour éviter la dégradation de l'image lors de rotations libres successives, l'algorithme conserve désormais en mémoire une copie de référence de l'image en niveaux de gris.

3.2. Binarisation et Segmentation Globale (Mathis BARBIER)

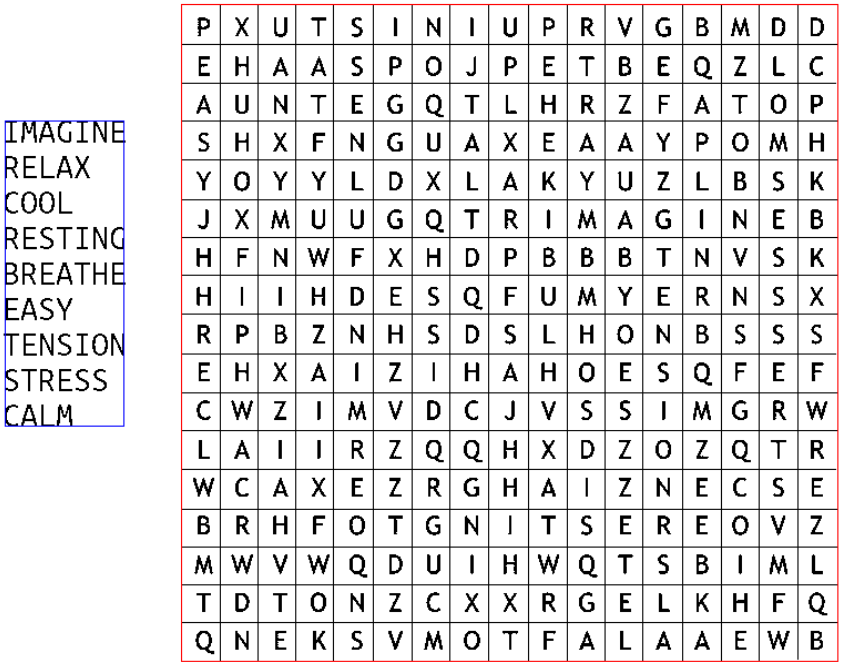
Pour préparer l'image à la reconnaissance, j'ai implémenté un algorithme de binarisation dynamique basé sur la méthode d'Otsu. Contrairement à un seuillage fixe qui échouerait sur des images mal éclairées, cette approche statistique calcule automatiquement le seuil optimal séparant le fond de l'encre.



Une fois l'image en noir et blanc, j'ai développé une analyse structurelle reposant sur des histogrammes de projection horizontaux et verticaux. En comptant le nombre de pixels noirs sur chaque axe, le programme repère les gouttières blanches (zones vides) séparant les différents blocs de texte. L'algorithme `find_layout_and_blocks` cherche le plus grand espacement sur l'axe X et sur l'axe Y pour déduire instantanément si la liste de mots est disposée à côté ou en dessous de la grille. Ensuite, j'ai conçu la fonction `crop_box` qui resserre mathématiquement les cadres de sélection au pixel près autour de l'encre, éliminant les marges inutiles.



Le défi algorithmique majeur est apparu sur les grilles de niveau 1 contenant des lignes de séparation bleues. Lors de la binarisation, ces lignes devenaient noires, ce qui faussait notre première approche consistant à compter les colonnes vides pour distinguer la grille de la liste de mots. Pour contourner ce problème, j'ai remplacé cette méthode par un calcul strict de la superficie de chaque bloc (Largeur × Hauteur), exploitant le fait géométrique que la zone de la grille est systématiquement plus vaste que celle de la liste.

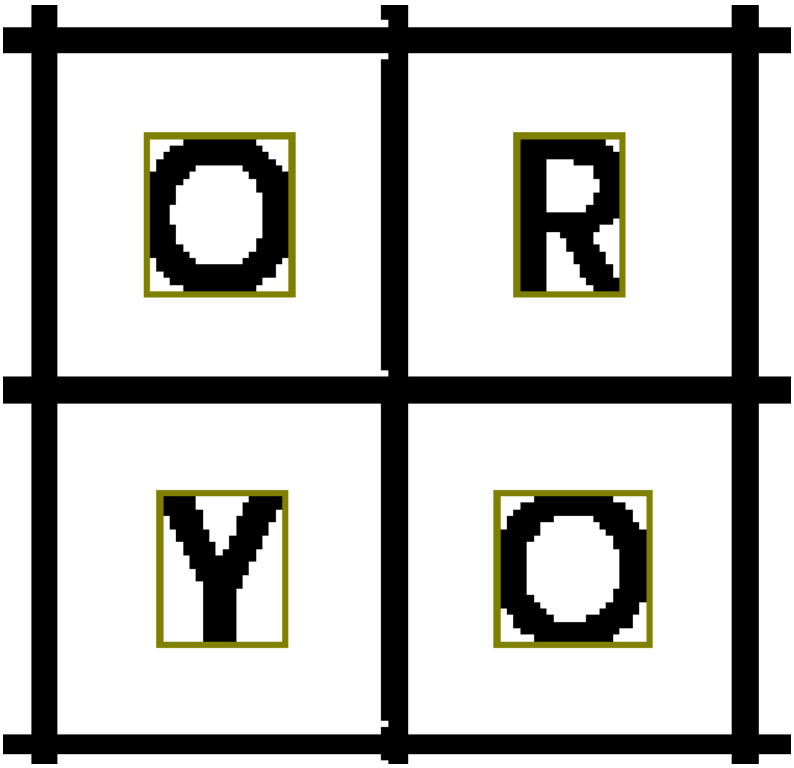


```c
void find_layout_and_blocks(int *vert_hist, int width, int *horiz_hist,
                            int height, int *b1_xmin, int *b1_xmax,
                            int *b1_ymin, int *b1_ymax, int *b2_xmin,
                            int *b2_xmax, int *b2_ymin, int *b2_ymax) {
    int threshold = 5;

    // 1. Find the largest hole on the X-axis (columns)
    int max_x_start = 0, max_x_width = 0, cur_x_start = 0, cur_x_len = 0;
    for (int x = 0; x < width; x++) {
        if (vert_hist[x] <= threshold) {
            if (cur_x_len == 0) cur_x_start = x;
            cur_x_len++;
        } else {
            if (cur_x_len > max_x_width) {
                max_x_width = cur_x_len;
                max_x_start = cur_x_start;
            }
            cur_x_len = 0;
        }
    }
```

3.3. Détection et Extraction des Lettres (Augustin SOURISSEAU)

Pour isoler chaque caractère à l'intérieur de la grille et de la liste de mots, j'ai mis en œuvre plusieurs algorithmes de détection adaptatifs (`segment_letters` et `segment_grid_letters_with_histogram`). Deux situations distinctes doivent être gérées : les grilles comportant des lignes de séparation explicites entre les cellules et les grilles sans quadrillage apparent.



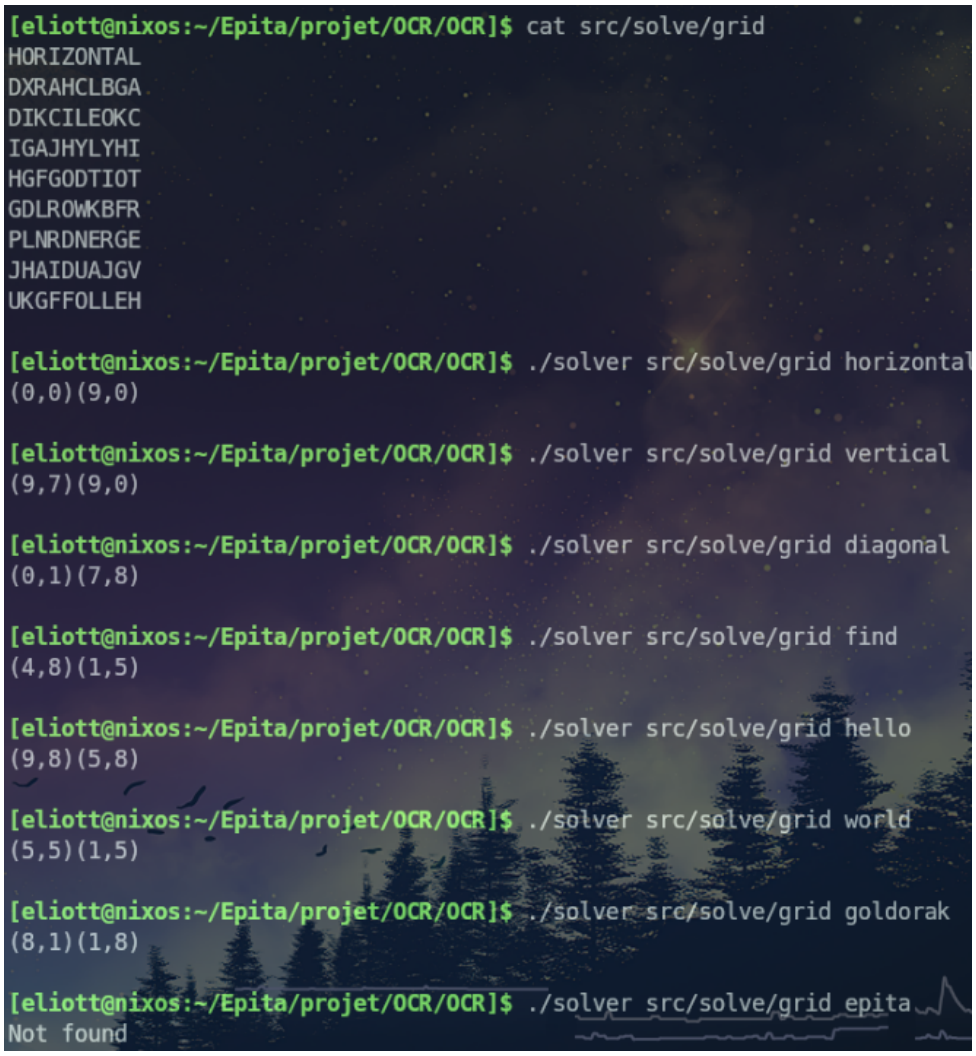
L'algorithme `has_grid` détermine le cas rencontré en analysant la densité des lignes : une ligne est considérée comme une bordure lorsqu'elle est composée d'au moins 80 % de pixels noirs. Lorsque la grille ne comporte pas de quadrillage, le programme utilise le même principe que pour

la détection des mots dans la liste, en supprimant ensuite le contour vert de segmentation pour éviter que l'algorithme `segment_letters` n'ignore certaines lettres (notamment T, H ou L). Pour les grilles quadrillées, la grille étant supposée ne pas dépasser 256 cellules, aucune allocation de mémoire supplémentaire n'est nécessaire. L'algorithme `compute_cell_margin` calcule l'épaisseur maximale des bordures d'une cellule et ajoute une marge d'un pixel pour éviter des erreurs lors de l'utilisation de `crop_box`. Une fois ce traitement effectué, chaque caractère isolé est individuellement sauvegardé au format `.png` dans un répertoire dédié nommé `image_storage`, générant ainsi un jeu de données prêt à être soumis au réseau de neurones.

Les principales difficultés concernaient la détection précise de chaque lettre sans confondre les limites du caractère avec les bordures de la cellule. Pour rendre le traitement plus robuste face au bruit numérique, j'ai ajusté finement plusieurs paramètres empiriques (`vert_limit`, `horiz_limit`, `LINE_GAP`, `CELL_MARGIN` et `threshold`).

3.4. Algorithme de Résolution – Solver (Eliott AVERTY)

La conception du solveur de mots cachés en ligne de commande m'a demandé de développer une recherche linéaire multidirectionnelle. Le programme charge le fichier texte de la grille et teste le mot saisi par l'utilisateur direction par direction (horizontale, verticale, diagonale) à partir du moment où une première lettre correspondante est trouvée. La difficulté principale résidait dans le parsing correct du tableau de lettres et la gestion des rebonds aux limites de la matrice. Le programme affiche ensuite sur la sortie standard la position de départ et d'arrivée selon le format strictement imposé `(x0,y0)(x1,y1)`.

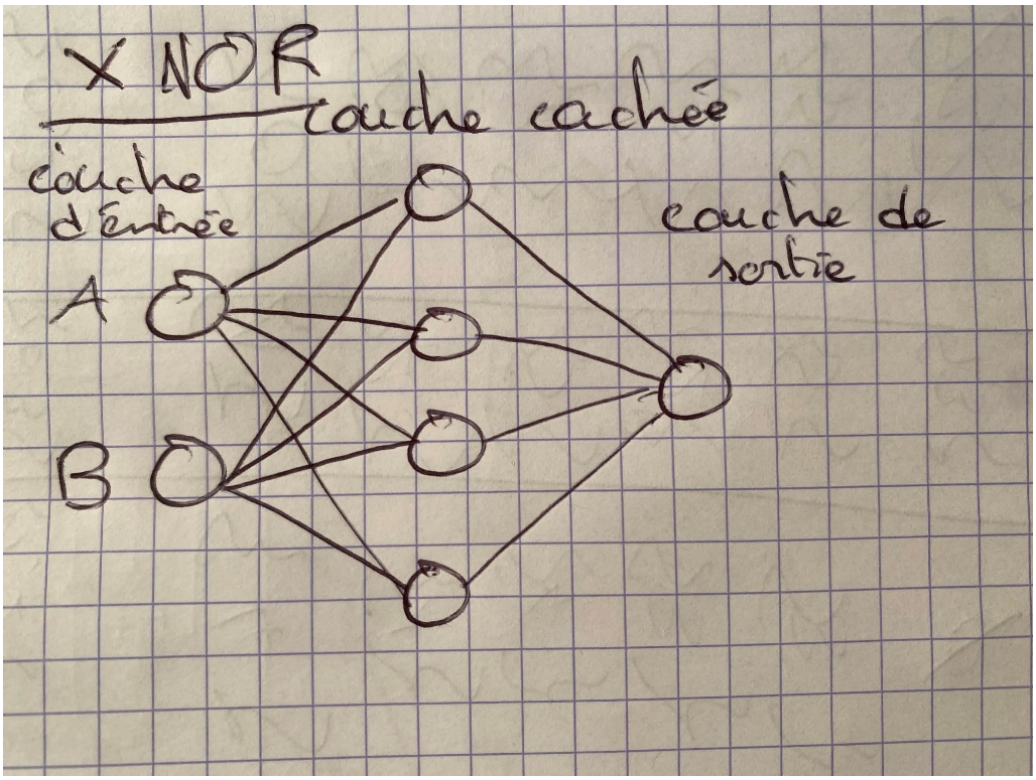


3.5. Fondations du Réseau de Neurones (Eliott AVERTY)

Pour la partie reconnaissance de caractères, j'ai développé un réseau de neurones intégralement *from scratch*. Afin de garantir l'adaptabilité du modèle au nombre de paramètres, de neurones et de couches, j'ai fait le choix algorithmique de vectoriser toutes les expressions mathématiques et de coder manuellement les implémentations des matrices et de leurs opérations. Le pipeline comprend la fonction d'initialisation du réseau, la fonction d'activation sigmoïde, la fonction *forward* permettant de passer une donnée à travers une couche, la fonction de coût calculant l'erreur, la fonction *backward* appliquant la rétropropagation des gradients, et la fonction *update* ajustant les poids selon un facteur d'apprentissage.

$$\text{Fonction Sigmoïde : } f(x) = \frac{1}{1+e^{-x}}$$

La compréhension mathématique profonde de ces mécanismes a constitué le problème majeur de ce développement. J'ai surmonté cette difficulté en m'appuyant sur des ressources théoriques externes pour assimiler la logique de la rétropropagation. Ces efforts ont abouti à l'implémentation réussie de l'algorithme d'apprentissage permettant de faire fonctionner un mini-réseau de 4 neurones validant la preuve de concept (PoC) sur la porte NXOR.



Conclusion Générale

En conclusion, cette première phase de développement du projet OCR Word Search Solver a permis à notre équipe de poser des fondations algorithmiques et logicielles solides. La conception d'un outil de reconnaissance optique de caractères en langage C exige une maîtrise rigoureuse de la mémoire et des structures de données, défis que nous avons relevés en développant des modules indépendants et fonctionnels.

L'ensemble des objectifs fixés par le cahier des charges pour cette première soutenance a été atteint avec succès. Nous disposons aujourd'hui d'une interface graphique capable de charger et de manipuler des images, d'un pipeline de traitement d'image robuste assurant la binarisation, la segmentation spatiale des blocs de texte et l'extraction individuelle des caractères. En parallèle, la logique mathématique de la résolution algorithmique a été validée à travers notre solveur en ligne de commande, tandis que notre réseau de neurones artificiels, codé intégralement *from scratch*, a prouvé son efficience sur la fonction logique NXOR.

Au-delà des aspects purement techniques, cette première période a été l'occasion de mettre en place une méthodologie de travail rigoureuse. L'utilisation d'outils de versioning, la division du travail selon les compétences de chacun, et le respect strict des normes de codage (limite des 80 colonnes, documentation en anglais) nous ont permis d'avancer de manière synchronisée sans compromettre la stabilité du code source.

Pour la soutenance finale de décembre, le défi majeur de notre équipe ne résidera plus dans la création isolée de ces briques algorithmiques, mais dans leur intégration au sein d'un pipeline automatisé. Il nous faudra implémenter la détection automatique de l'angle de rotation (*deskewing*), procéder à la phase d'apprentissage massif de notre réseau de neurones sur des jeux de données réels, et enfin, relier la reconnaissance visuelle au solveur pour afficher directement la grille résolue sur notre interface graphique. Forts des bases établies lors de ce premier sprint, nous abordons cette seconde phase avec une architecture claire et une vision précise des étapes

restantes.

Bibliographie et Ressources

Ouvrages et documents de référence

— BOUCHET, David. *OCR Word Search Solver — Cahier des charges*, Projet EPITA S3, 2026-2027.
— OTSU, Nobuyuki. *A Threshold Selection Method from Gray-Level Histograms*, IEEE Transactions on Systems, Man, and Cybernetics, 1979. (Base théorique de l'algorithme de binarisation dynamique).
— Concepts de *Document Layout Analysis* : algorithme *Recursive X-Y Cut* (R-XYC) et principes de *Bounding Box* pour la segmentation structurelle de l'image.

Documentations techniques officielles

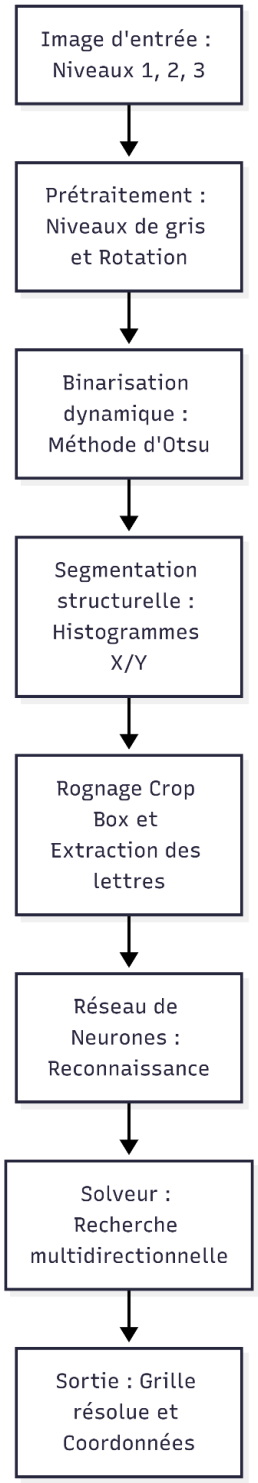
— **SDL Wiki** : Documentation des structures `SDL_Surface`, des fonctions de chargement `IMG_Load`, des formats de pixels `SDL_PixelFormat`, et de l'accès mémoire direct sécurisé via `SDL_LockSurface`.
— **Documentation GTK 3 Reference** : Utilisation de `GtkDrawingArea` pour le rendu graphique personnalisé et de `GtkCssProvider` pour l'intégration de feuilles de style.
— **GNU Project** : *GNU Make Manual* pour l'automatisation de la compilation du projet en C.
— **Git Project** : *Pro Git* pour la gestion des branches, des versions et des fusions.

Ressources d'apprentissage et forums

— **Machine Learnia** (Chaîne YouTube) : Séries de tutoriels sur le Deep Learning, essentiels pour la compréhension mathématique des réseaux de neurones, de la fonction de coût et de la rétropropagation du gradient.
— **OpenClassrooms** : Cours sur le langage C, la gestion de la mémoire et l'utilisation de la bibliothèque SDL2.
— **Stack Overflow** : Résolution de problèmes ponctuels d'implémentation algorithmique et de gestion des pointeurs en C.

Annexes

— Organigramme du pipeline :



— Apprentissage et réponse de l'IA :

